

Ambulance Dispatch System Design

D795: Applied Algorithms & Reasoning

I plan to build a Python application that uses simulation data to decide which ambulance should respond to each emergency call. The application will process calls by priority, compare travel times, and record each dispatch. Both prototypes will follow the same design, with the routing algorithm being the main difference, as required by the assessment (Western Governors University [WGU], n.d.-a).

Overall Operation of the Application

When the application starts, it will load the four simulation files and check that the data can be used together. It will then organize the calls by priority. For each call, the application will find the fastest route from each ambulance's staging location to the call location. It will include traffic delays when comparing the routes and select the ambulance with the lowest total travel time. If an ambulance is already at the call location, its travel time will be zero. For equal travel times, I plan to use ambulance number as a consistent way to break the tie.

After selecting an ambulance, the application will create a dispatch record with the call ID, call type, call location, selected ambulance, route to the call location, and time to the call location. It will append the record to `/var/log/ambulance_call_log.csv` as comma-separated name-value pairs. Once the record is saved, the ambulance will return to its staging location in the simulation and be available for the next call. This process will continue until every call has been dispatched (WGU, n.d.-a).

I will keep the routing function separate from file processing, priority handling, and logging so that I can change the algorithm for the second prototype. Each prototype will also use the required embedded counters to measure and display the total time spent finding routes for all

calls. This measures the algorithm's execution time, which is different from the simulated ambulance travel time (WGU, n.d.-a).

Processing the Simulation Files

I plan to read the files with Python's csv module and use their column headers to identify each value. The application will convert priorities to integers and route measurements to numeric values. Before dispatching, it will check for missing fields, invalid numbers, duplicate call IDs, unknown call types, and locations that do not appear in the network. If a required value is invalid, it will display the file and row involved so the data can be corrected before the simulation runs.

The ambulance.csv file contains three ambulances and their staging locations. I will store each ambulance number with its starting location. The call_priority.csv file assigns a priority to each of 20 call types. I will store these pairs in a dictionary so the application can look up the priority of each call. The calls.csv file contains 100 calls with a Call ID, Location, and Call Type. I will preserve the order of these rows as the order received because the file does not include arrival timestamps (WGU, n.d.-b).

The location_network.csv file contains Start, End, Distance, Travel Time, and Traffic Delay columns. I will represent the locations as points in a graph and each row as a directed connection between two locations. Direction matters because the values for a return trip can differ. The cost of each connection will be Travel Time plus Traffic Delay, and a route's total cost will be the sum of its connections. Distance will remain available in the data, but dispatch decisions will use travel time. For example, the connection from Intersection B to Address 456 Oak St has a travel time of 1.16 and a delay of 2, giving a combined cost of 3.16 in the simulation's time units (WGU, n.d.-b).

Managing Call Priorities

I will treat priority 1 as the highest priority, followed by priority 2 and then priority 3. Each call will receive its priority from its call type. Within each priority group, calls will stay in the order they appeared in calls.csv. I plan to arrange the calls using priority first and original row position second. This follows the requirement to handle the highest-priority calls first while preserving the order received within that priority (WGU, n.d.-a).

For example, call 1 is a Stroke with priority 1, call 2 is a Minor Car Accident with priority 2, and call 3 is a House Fire with priority 1. Call 1 will be processed before call 3, and both will be processed before call 2. All remaining priority 1 calls will also be handled before priority 2 calls begin. The selected ambulance and route will not change a call's priority. Both prototypes will use this same ordering so their routing algorithms can be compared using the same sequence of calls (WGU, n.d.-b).

References

Western Governors University. (n.d.-a). *D795 Applied Algorithms & Reasoning performance assessment* [Course assessment].

Western Governors University. (n.d.-b). *D795 Applied Algorithms & Reasoning simulation files* [Data set].